Projeto - Cidades ESGInteligentes

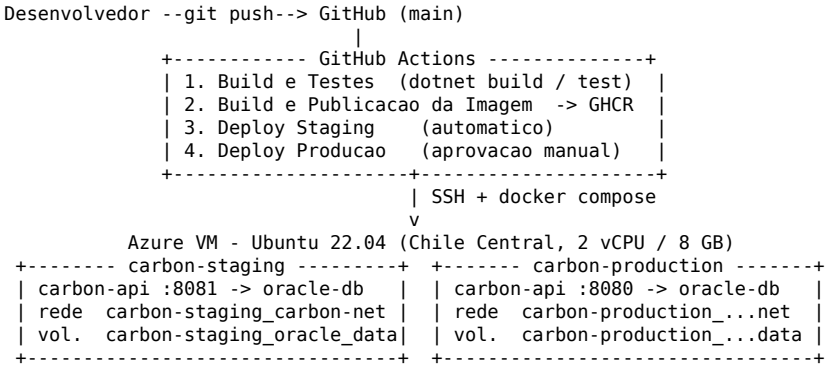
Carbon API — Gestão de Emissões de Carbono
Documentação técnica de DevOps: containerização, orquestração e CI/CD

Integrante	RM
Gabriel Alves Oliveira	RM562365
Marcos Antão Fernandes Junior	RM563156
Marcos Eduardo Alves de Oliveira Freitas	RM555914
Danilo Ferreira Chaves	RM565765
Hevilyn Romano Garcia	RM566426

FIAP — Análise e Desenvolvimento de Sistemas
Fase: Navegando pelo mundo DevOps
Repositório: https://github.com/GabrielOlliver/carbon-api-devops

1. Visão geral e arquitetura

A Carbon API é uma API RESTful em .NET 8 para gestão de emissões, reduções e compensações de carbono (tema ESG), com persistência em Oracle 23ai. Neste projeto ela foi adaptada para um ambiente de produção simulado: a aplicação é containerizada, orquestrada com Docker Compose e entregue por um pipeline de CI/CD no GitHub Actions que faz build, testes e deploy automatizado em dois ambientes (staging e produção) numa máquina virtual Linux no Microsoft Azure.



Cada ambiente é um projeto Docker Compose isolado (containers, rede, volume, portas e segredos próprios) e ambos executam exatamente a mesma imagem, identificada pelo SHA do commit — o que foi validado em staging é o que chega à produção.

Componente	Tecnologia	Papel
Aplicação	.NET 8 / ASP.NET Core / EF Core 8	API REST com JWT, Swagger e health checks
Banco de dados	Oracle 23ai Free (gvenzl/oracle-free:23-slim)	Persistência, com volume Docker
Registro de imagens	GitHub Container Registry (GHCR)	Armazena as imagens versionadas por commit
CI/CD	GitHub Actions + Environments + Secrets	Build, testes, publicação e deploy
Infraestrutura	Azure VM Ubuntu 22.04 + cloud-init	Host Docker de staging e produção

2. Pipeline de CI/CD

Ferramenta: GitHub Actions. O pipeline está em **.github/workflows/ci-cd.yml** e a lógica de deploy num workflow reutilizável, **.github/workflows/deploy.yml**, chamado uma vez para cada ambiente — a mesma receita serve para staging e produção, mudando apenas parâmetros e segredos.

Gatilhos: todo push na branch main executa CI + CD; pull requests executam apenas o CI (sem deploy); e há execução manual (workflow_dispatch).

#	Job	Lógica
1	Build e Testes	dotnet restore → dotnet build -c Release → dotnet test (10 testes xUnit). Publica o relatório .trx como artifact, mesmo em caso de falha.
2	Build e Publicação da Imagem	Só roda se o job 1 passar (needs). Build da imagem com cache de camadas (type=gha) e push no GHCR com as tags <sha-do-commit> e latest.
3	Deploy Staging	Automático após o job 2. SSH na VM, cópia do compose, geração do .env a partir dos secrets do environment staging, pull da imagem e docker compose up (porta 8081). Smoke test em /health/ready.
4	Deploy Produção	Só após staging com sucesso e **aprovação manual** (environment production com required reviewers). Mesmo processo, porta 8080, segredos próprios.

Gestão de segredos

Nenhuma senha fica no repositório. O .env local é ignorado pelo Git e pelo Docker; no pipeline, o arquivo .env de cada ambiente é gerado na VM a partir dos GitHub Secrets: VM_SSH_KEY no nível do repositório e ORACLE_PASSWORD, APP_USER_PASSWORD e JWT_SECRET em cada environment — senhas diferentes para staging e produção. VM_HOST e VM_USER são variables (não sensíveis).

Smoke test

O deploy só é considerado bem-sucedido quando /health/ready responde Healthy — esse endpoint verifica que a API está no ar e consegue consultar o banco. Se não responder em até 10 minutos, o job imprime os logs dos containers e falha.

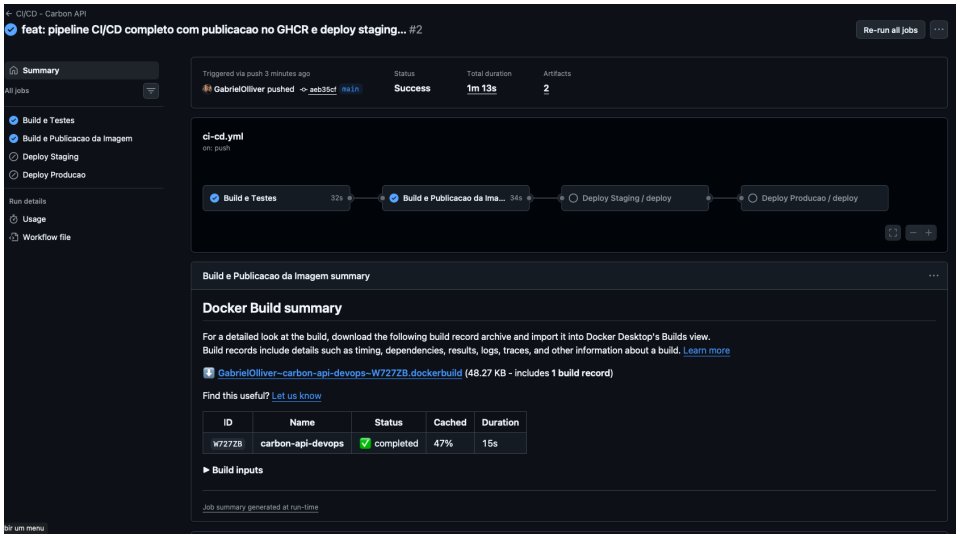


Figura 1 — Pipeline com os jobs encadeados (antes da configuração da VM, deploys pulados).

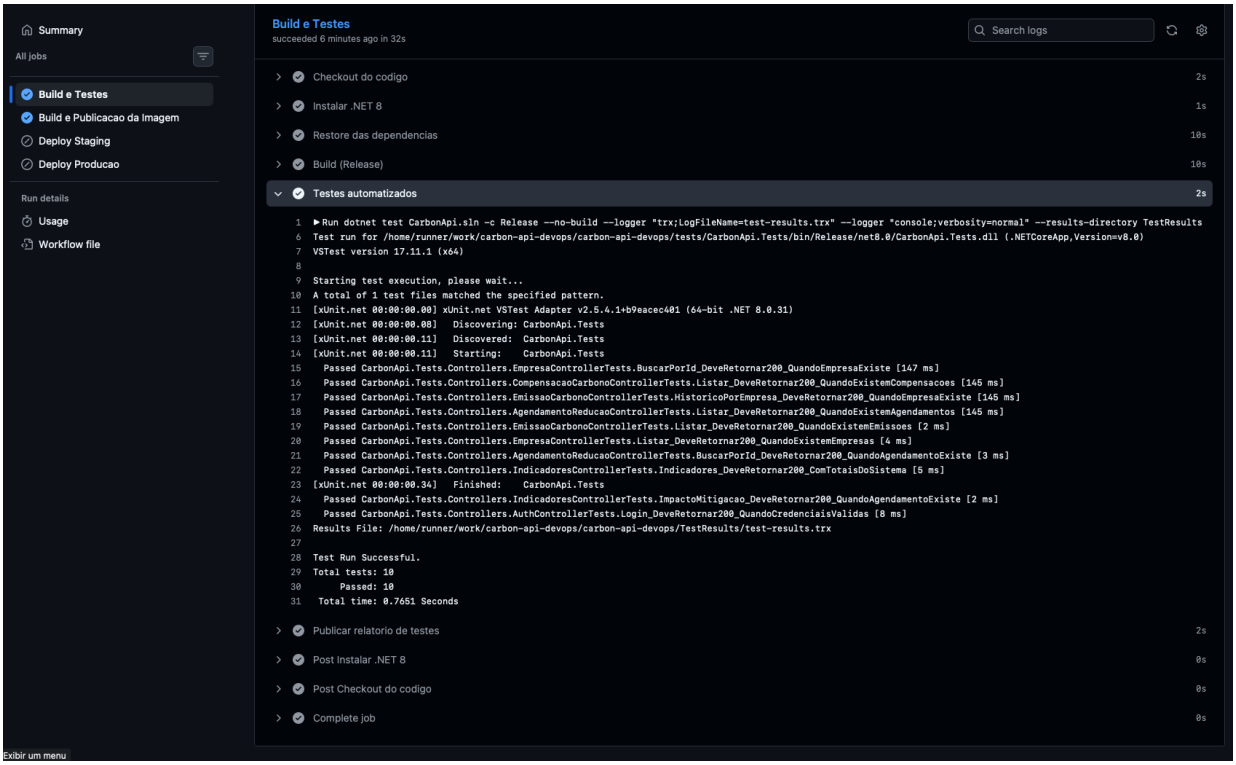


Figura 2 — Job Build e Testes: 10 testes aprovados no runner do GitHub.



Figura 3 — Artifacts publicados: relatório de testes (test-results) e build record.

3. Docker: arquitetura, comandos e imagem

Dockerfile (multi-stage)

```
# ================================================================
# ESTAGIO 1 - BUILD: imagem com o SDK completo (~800 MB), so para compilar
# ================================================================
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# 1) Copia SOMENTE os arquivos de dependencias e faz o restore.
#    Enquanto o .csproj nao mudar, o Docker reaproveita esta camada do cache
#    e o restore (lento) nao roda de novo a cada alteracao de codigo.
COPY global.json ./
COPY src/CarbonApi/CarbonApi.csproj src/CarbonApi/
RUN dotnet restore src/CarbonApi/CarbonApi.csproj

# 2) Agora copia o codigo-fonte e publica em modo Release.
COPY src/ src/
RUN dotnet publish src/CarbonApi/CarbonApi.csproj \
        -c Release \
        -o /app/publish \
        --no-restore \
        /p:UseAppHost=false

# ================================================================
# ESTAGIO 2 - RUNTIME: imagem enxuta, so com o runtime do ASP.NET
# ================================================================
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# curl e usado pelo HEALTHCHECK (a imagem base nao traz nenhum cliente HTTP)
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*

# Copia apenas o resultado do publish - o SDK e o codigo-fonte ficam para tras
COPY --from=build /app/publish .

ENV ASPNETCORE_HTTP_PORTS=8080 \
    ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080

# Roda como usuario sem privilegios (o usuario "app" ja existe na imagem base)
USER $APP_UID

# O Docker marca o container como healthy/unhealthy consultando a API
HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
  CMD curl -fsS http://localhost:8080/health || exit 1

ENTRYPOINT ["dotnet", "CarbonApi.dll"]
```

Estratégia	Benefício
Multi-stage (sdk → aspnet)	Imagem final com ~98 MB de conteúdo, sem SDK nem código-fonte
Restore em camada separada	Alterações no código não refazem o restore (cache do Docker)
Usuário não-root (USER $APP_UID)	Reduz o impacto de uma eventual invasão do container
HEALTHCHECK em /health	O Docker sabe se a API realmente responde
.dockerignore	Exclui bin/obj, .git, testes e segredos (.env) do contexto de build
Configuração por variáveis de ambiente	Mesma imagem em qualquer ambiente (12-Factor App)

docker-compose.yml

```
# Nome padrao do projeto. Em staging/producao o pipeline sobrescreve com -p
# (carbon-staging / carbon-prod), isolando containers, rede e volumes de cada ambiente.
name: carbon-api

services:

  # ------------------------------------------------------------------
  # Banco de dados Oracle 23ai Free
  # ------------------------------------------------------------------
  oracle-db:
    image: gvenzl/oracle-free:23-slim
    restart: unless-stopped
    environment:
      ORACLE_PASSWORD: ${ORACLE_PASSWORD:?defina ORACLE_PASSWORD no .env}
      APP_USER: ${APP_USER:?defina APP_USER no .env}
      APP_USER_PASSWORD: ${APP_USER_PASSWORD:?defina APP_USER_PASSWORD no .env}
    ports:
      # Publicado SOMENTE em 127.0.0.1: acessivel da propria maquina
      # (DBeaver, Rider), nunca exposto para a internet.
      - "127.0.0.1:${DB_PORT:-1521}:1521"
    volumes:
      - oracle_data:/opt/oracle/oradata   # dados sobrevivem a recriacao do container
    networks:
      - carbon-net
    healthcheck:
      test: ["CMD", "healthcheck.sh"]
      interval: 15s
      timeout: 10s
      retries: 20
      start_period: 90s

  # ------------------------------------------------------------------
  # API .NET 8
  # ------------------------------------------------------------------
  carbon-api:
    # Localmente: construida a partir do Dockerfile.
    # No deploy: API_IMAGE aponta para a imagem publicada pelo pipeline no GHCR.
    image: ${API_IMAGE:-carbon-api:local}
    build:
      context: .
      dockerfile: Dockerfile
    restart: unless-stopped
    environment:
      # "oracle-db" e o nome do servico: o DNS interno da rede do Compose resolve para o container do banco
      ConnectionStrings__OracleConnection: "User Id=${APP_USER};Password=${APP_USER_PASSWORD};Data Source=oracle-db:1521/FREEPD
      Jwt__Secret: ${JWT_SECRET:?defina JWT_SECRET no .env}
      Jwt__Issuer: ${JWT_ISSUER:-CarbonApi}
      Jwt__Audience: ${JWT_AUDIENCE:-CarbonApiUsers}
      Jwt__ExpirationHours: ${JWT_EXPIRATION_HOURS:-24}
      ASPNETCORE_ENVIRONMENT: ${ASPNETCORE_ENVIRONMENT:-Production}
    ports:
      - "${API_PORT:-8080}:8080"
    networks:
      - carbon-net
    depends_on:
      oracle-db:
        condition: service_healthy   # so sobe a API quando o Oracle estiver pronto

networks:
  carbon-net:
    driver: bridge

volumes:
  oracle_data:
```

Comandos utilizados

Comando	Finalidade
docker build -t carbon-api:local .	Construir a imagem localmente
docker run -d -p 8080:8080 -e Jwt__Secret=... carbon-api:local	Testar a imagem isolada
docker compose up -d --build	Subir API + Oracle (rede, volume, healthchecks)
docker compose ps / logs -f carbon-api	Acompanhar estado e logs

Comando	Finalidade
docker compose down (sem -v)	Derrubar mantendo os dados (teste de persistência)
docker compose -p carbon-staging up -d --no-build	Deploy isolado por ambiente na VM (via pipeline)
az vm create ... --custom-data infra/cloud-init.yml	Criar a VM no Azure já com Docker instalado

```
→  CarbonApi git:(main) ✗ docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE      CREATED         STATUS
          PORTS
carbon-api-carbon-api-1     carbon-api:local            "dotnet CarbonApi.dll"   carbon-api   6 minutes ago   Up 5 minute
s (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
carbon-api-oracle-db-1      gvenzl/oracle-free:23-slim  "container-entrypoin…"  oracle-db    6 minutes ago   Up 6 minute
s (healthy)   127.0.0.1:1521->1521/tcp
```

Figura 4 — Stack local: API e Oracle healthy; Oracle exposto apenas em 127.0.0.1.

```
→  CarbonApi git:(main) ✗ docker compose logs carbon-api | grep -i migration
carbon-api-1  |       SELECT t.table_name FROM user_tables t WHERE t.table_name = N'__EFMigrationsHistory'
carbon-api-1  |       "__EFMigrationsHistory" (
carbon-api-1  |           "MigrationId" NVARCHAR2(150) NOT NULL,
carbon-api-1  |           CONSTRAINT "PK___EFMigrationsHistory" PRIMARY KEY ("MigrationId")
carbon-api-1  |       SELECT t.table_name FROM user_tables t WHERE t.table_name = N'__EFMigrationsHistory'
carbon-api-1  |       SELECT "MigrationId", "ProductVersion"
carbon-api-1  |       FROM "__EFMigrationsHistory"
carbon-api-1  |       ORDER BY "MigrationId"
carbon-api-1  | info: Microsoft.EntityFrameworkCore.Migrations[20402]
carbon-api-1  |       Applying migration '20260619054402_InitialCreate'.
carbon-api-1  |       INSERT INTO "__EFMigrationsHistory" ("MigrationId", "ProductVersion")
carbon-api-1  |       Migrations aplicadas com sucesso.
→  CarbonApi git:(main) ✗ curl http://localhost:8080/health/ready
Healthy%
→  CarbonApi git:(main) ✗
```

Figura 5 — Migrations aplicadas automaticamente e /health/ready = Healthy.

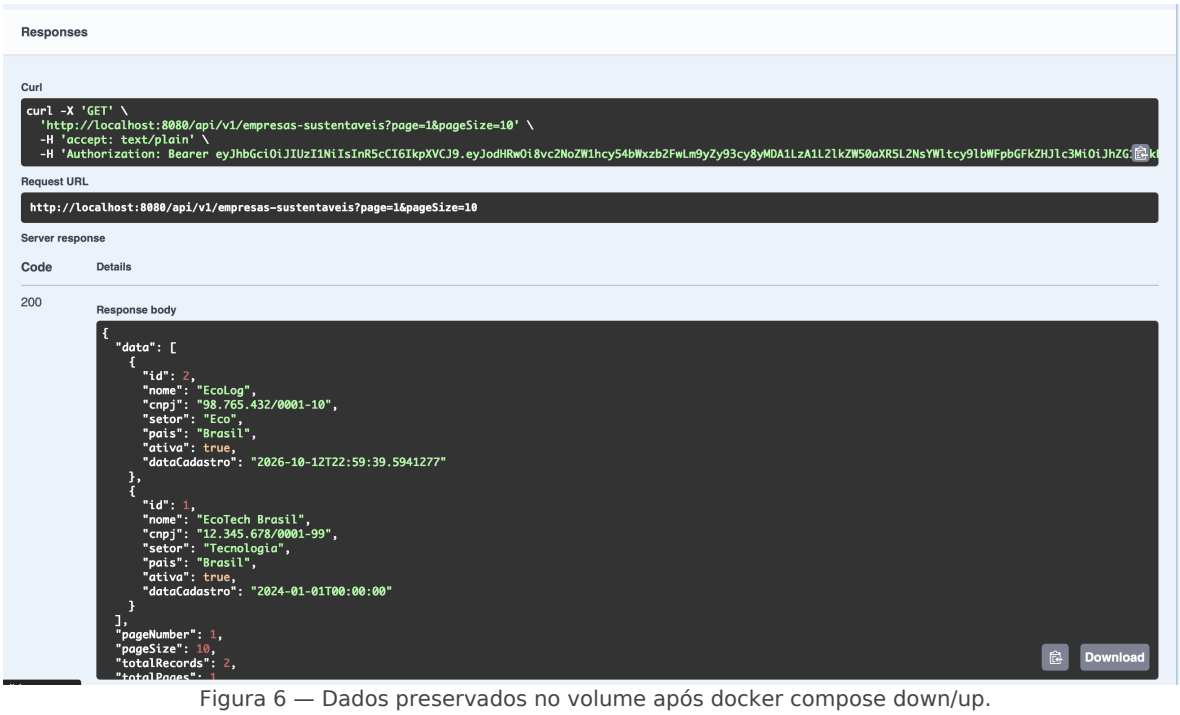


Figura 6 — Dados preservados no volume após docker compose down/up.

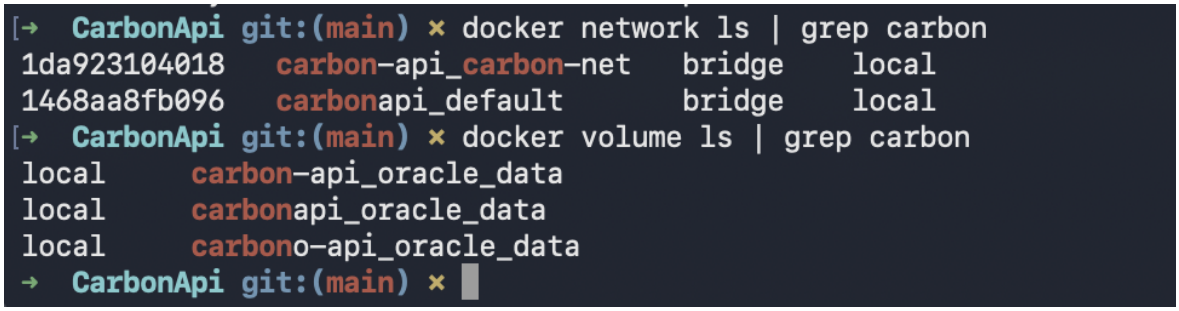


Figura 7 — Rede bridge e volume nomeado criados pelo Compose.

4. Pipeline rodando: build, testes e deploy

```
→  CarbonApi git:(main) ✗ ssh -i ~/.ssh/carbon_deploy azureuser@$IP "docker --version && docker compose version && free
-h"
The authenticity of host '68.211.126.191 (68.211.126.191)' can't be established.
ED25519 key fingerprint is: SHA256:ymX94lfq5j/NSMs8JUoUDgM799gzCz9bYPYprRk+gZM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '68.211.126.191' (ED25519) to the list of known hosts.
Docker version 29.9.0, build f415da8
Docker Compose version v5.6.0
               total        used        free      shared  buff/cache   available
Mem:           7.8Gi       327Mi       6.1Gi       3.0Mi       1.4Gi       7.2Gi
Swap:          4.0Gi          0B       4.0Gi
→  CarbonApi git:(main) ✗
```

Figura 8 — VM no Azure provisionada por cloud-init: Docker, Compose, 7,8 GB RAM + 4 GB swap.

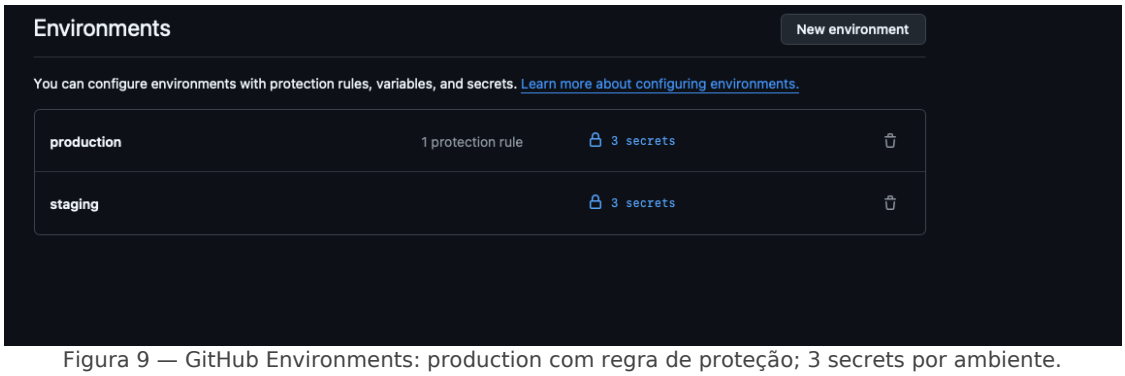


Figura 9 — GitHub Environments: production com regra de proteção; 3 secrets por ambiente.

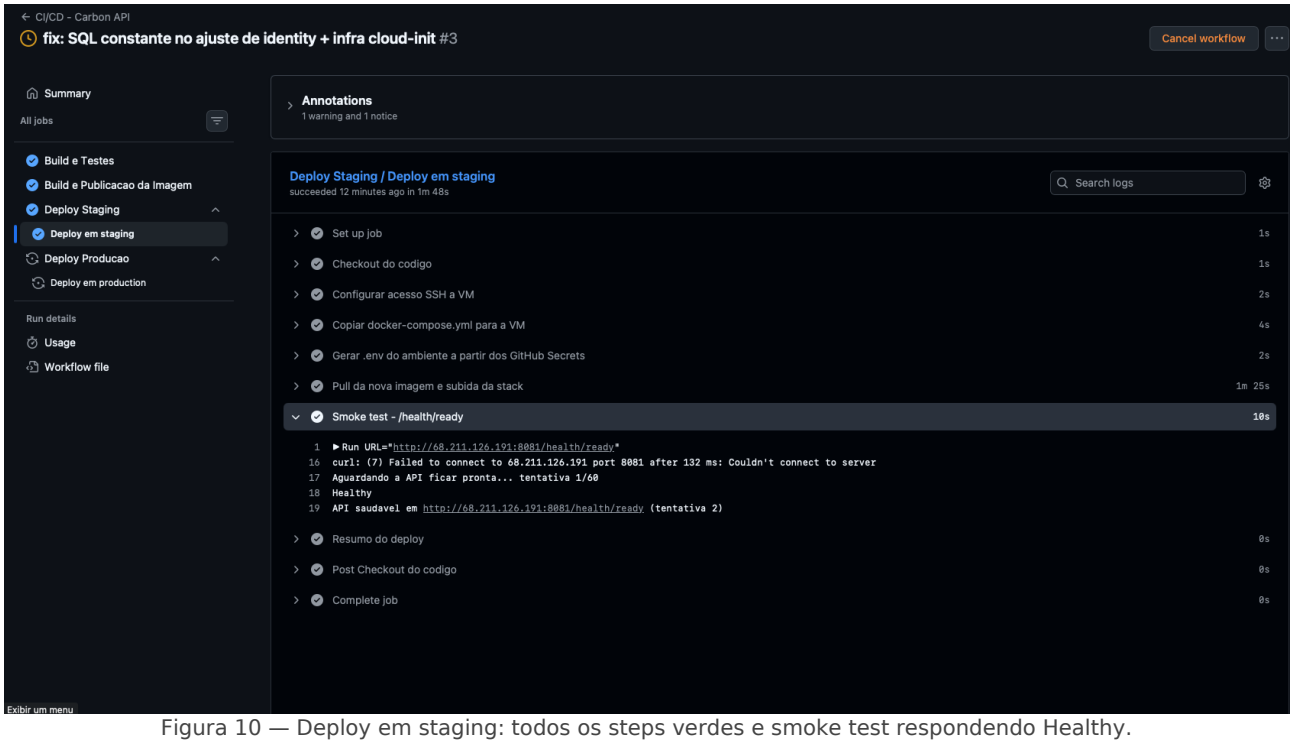


Figura 10 — Deploy em staging: todos os steps verdes e smoke test respondendo Healthy.

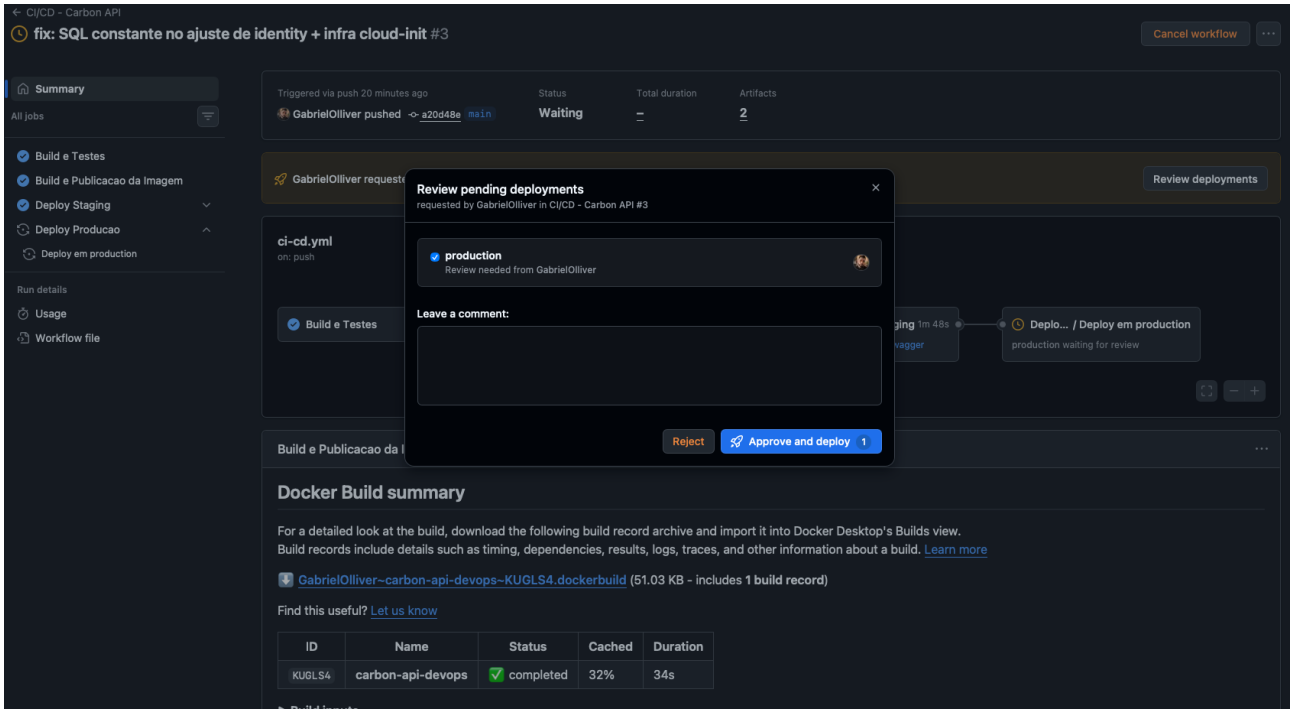


Figura 11 — Aprovação manual exigida antes do deploy em produção.

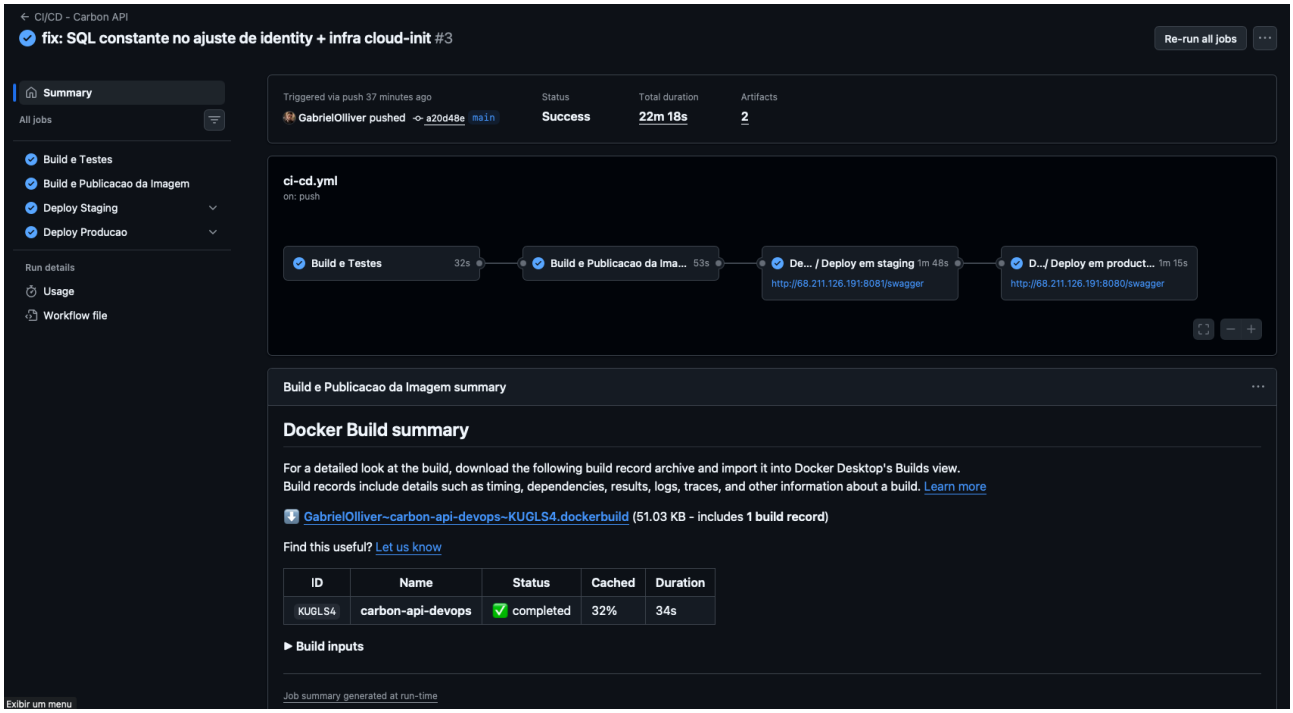


Figura 12 — Execução do pipeline completo: build, imagem, staging e produção.

5. Ambientes staging e produção funcionando

Ambiente	URL	ASPNETCORE_ENVIRONMENT	Proteção
Staging	http://68.211.126.191:8081/swagger	Staging	Deploy automático
Produção	http://68.211.126.191:8080/swagger	Production	Aprovação manual

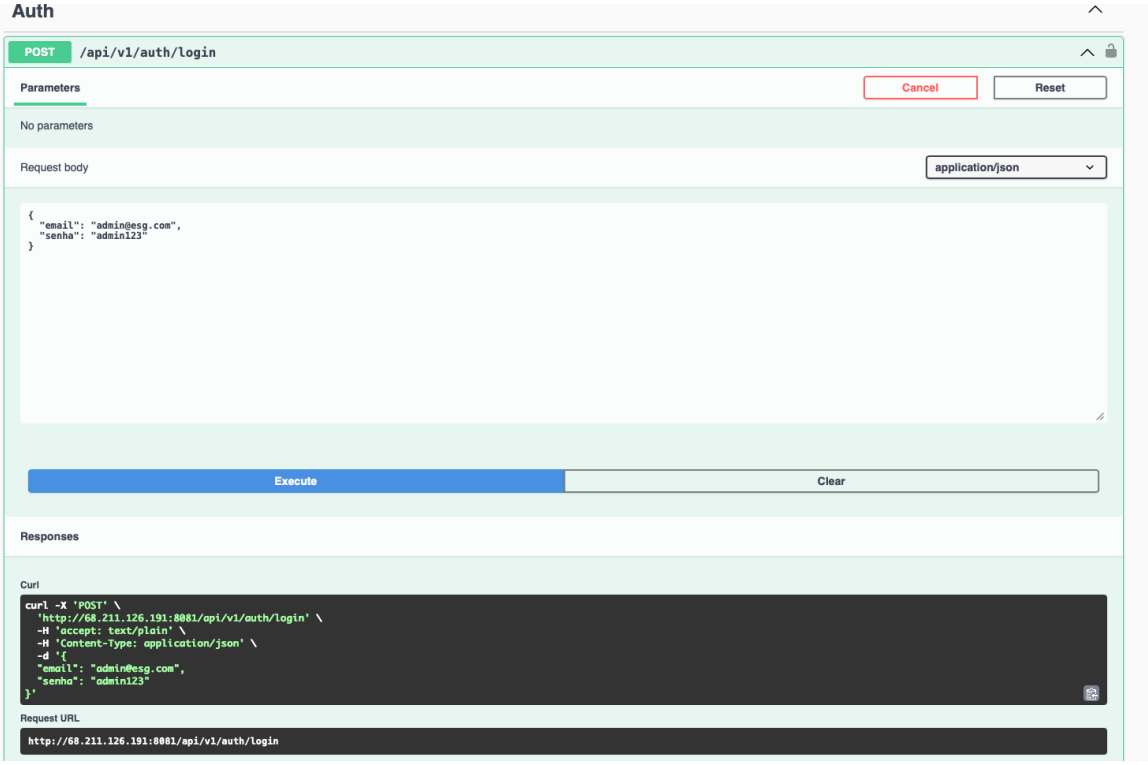


Figura 13 — Staging (porta 8081): login executado no Swagger publicado na VM.

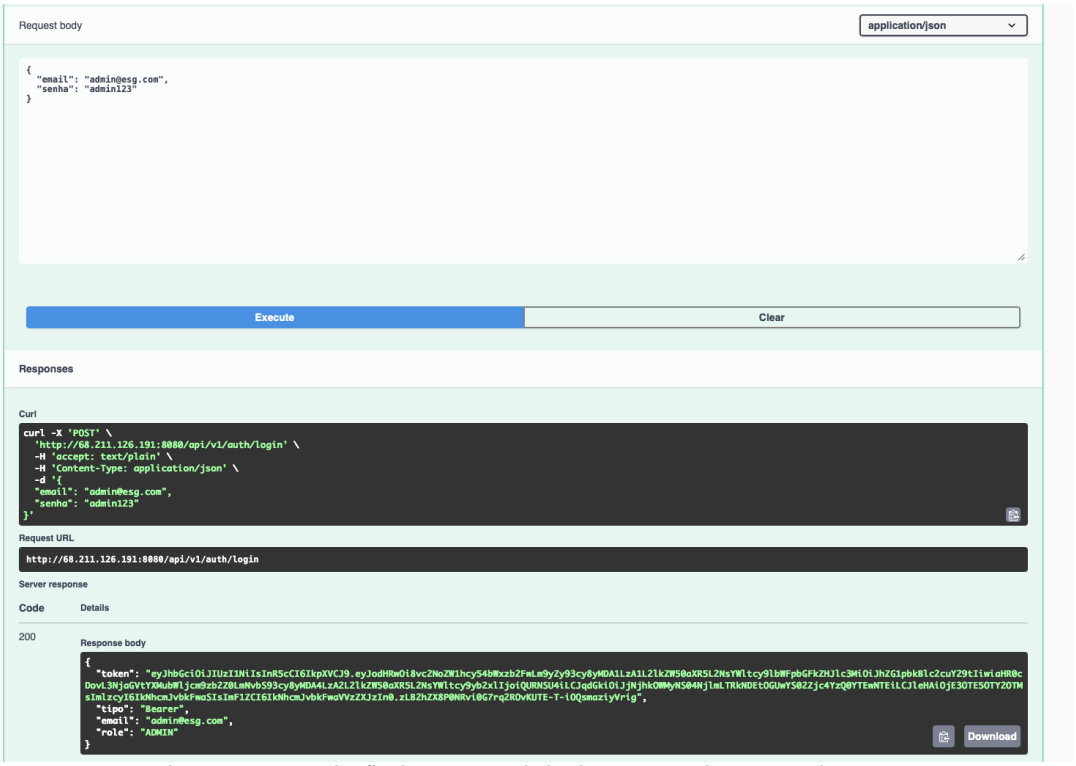


Figura 14 — Produção (porta 8080): login retornando 200 e token JWT.

6. Desafios encontrados e soluções

Desafio	Como foi resolvido
Senha do banco e chave JWT gravadas no appsettings.json e no docker-compose.yml	Segredos movidos para .env (fora do Git), User Secrets para execução local e GitHub Secrets no pipeline. Todos os valores foram trocados, pois já haviam sido expostos. A API passou a falhar na inicialização se o Jwt:Secret não existir (fail fast).
Build alertando vulnerabilidade (NU1902) em System.IdentityModel.Tokens.Jwt 7.0.3	Atualização dos pacotes JWT para 7.1.2; build passou a terminar com 0 avisos.
Erro 'failed to connect to the docker API' ao construir a imagem	O daemon não estava em execução: Docker Desktop iniciado e verificado com docker version.
Container novo subia com o banco vazio (migrations eram aplicadas manualmente)	Database.Migrate() na inicialização da API, com 10 tentativas a cada 5 s enquanto o Oracle termina de subir.
Erro 500 ao criar empresa: ORA-00001 (ID 1 duplicado) — o seed insere Id=1, mas a coluna IDENTITY do Oracle continuava começando em 1	Diagnóstico pelos logs do container. Após as migrations, ALTER TABLE ... IDENTITY (START WITH LIMIT VALUE) reposiciona a sequência para MAX(ID)+1 — idempotente.
Analisador do EF Core apontando risco de SQL injection no ExecuteSqlRaw	SQL reescrito como texto constante, sem interpolação de strings.
Azure for Students: RequestDisallowedByAzure em brazilsouth	Política de regiões da assinatura; VM criada em chilecentral, região liberada.
SkuNotAvailable: tamanho Standard_B2ms sem capacidade na região	Script em loop testando tamanhos equivalentes (2 vCPU / 8 GB) até encontrar um disponível.
Dois ambientes na mesma VM causariam conflito de nomes e portas	container_name removido; cada ambiente roda como projeto Compose próprio (-p) com portas parametrizadas (8081/8080) e Oracle publicado só em 127.0.0.1.
Desenvolvimento em Mac M1 (ARM) e VM Linux x86	A imagem de deploy é construída no runner x86 do GitHub Actions, não na máquina local.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑
Dockerfile funcional	☑
docker-compose.yml ou arquivos Kubernetes	☑
Pipeline com etapas de build, teste e deploy	☑
README.md com instruções e prints	☑
Documentação técnica com evidências (PDF ou PPT)	☑
Deploy realizado nos ambientes staging e produção	☑